

## ASSIGNMENT 7

### REPORT

#### Introduction

CUDA is a platform made by NVIDIA which allows GPU to do general purpose computing. Normally CPU works one by one.

In this assignment, we tried some basic CUDA programs like sum, merge sort and vector addition. Also we checked performance.

#### Problem 1: Sum of First N Integers

##### Objective

To find sum of first N numbers using CUDA by using different threads.

##### Approach

First we created an array of size 1024 and filled it from 1 to N.  
Then data is copied from CPU to GPU.

We used 2 threads:

Thread 0 → calculates sum using loop

Thread 1 → calculates sum using formula

After that result is copied back to CPU.

##### Inference

Both outputs are same, so program is correct.

But formula method is faster because it directly gives result.

Loop method takes more time since it runs N times.

So here we see that algorithm also matters, not only parallel.

#### Problem 2: Merge Sort (CPU vs CUDA)

##### Objective

To implement merge sort in three ways:

- Serial (recursive)
- CPU pipelined (iterative)
- CUDA (parallel)

##### Approach

###### (a) Serial Merge Sort

Uses recursion. It divides array and merges. Takes more time because of function calls.

###### (b) CPU Pipelined Merge Sort

Uses bottom-up approach. No recursion so little faster than serial.

###### (c) CUDA Merge Sort

Uses GPU threads to merge parts of array in parallel. Uses grid and block for execution.

## Results

Serial Merge Sort Time = 0.132000 ms  
CPU Pipelined Merge Sort Time = 0.112000 ms  
CUDA Merge Sort Time = 25.470049 ms

## Inference

CUDA is parallel but still slower here.  
Reason is input size is small (1000), so overhead like memory copy and kernel launch is high.

CPU methods are faster for small data.  
But for big data, CUDA will perform better.

## Problem 3: Vector Addition & Bandwidth Analysis

### Objective

To perform vector addition using CUDA and check memory bandwidth.

### Approach

Two arrays are created and added using CUDA kernel.  
We also used device properties to calculate theoretical bandwidth.

Measured bandwidth is calculated using:

- number of bytes read and written
- execution time

### Inference

Theoretical bandwidth is very high but measured bandwidth is low.

This happens because:

- memory latency
- kernel overhead
- not perfect memory access

So practical performance is always less than theoretical.

### Conclusion

In this assignment we learned how CUDA works and how GPU can be used for parallel computing.

We saw that:

- GPU is very powerful for large data
- But for small input, CPU can be faster
- Algorithm also plays important role

Overall CUDA is good for performance, but depends on how we use it.

Iterative Sum = 524800

Formula Sum = 524800

nvcc warning : Support for offline compilation

C[0] = 3.000000

Kernel Time = 12.173536 ms

Theoretical Bandwidth = 320.063995 GB/s

Measured Bandwidth = 1.033628 GB/s